

Mess Queue Management (MQM) — Project Report

Project: Mess Queue Management

Repository: github.com/the-robotronics-club/MQM

Team: The Robotronics Club

Date: September 2026

1. Introduction

Every meal hour at our institute, students walk to the mess, glance at the queue, and make a guess: wait or come back later. There is no information available before they arrive. If the queue is long, they've already wasted the trip.

MQM fixes this by putting a camera above the mess entrance, running person detection on each frame, and publishing live queue length and seating occupancy to a web dashboard that any student can check from their phone.

The system is a wall-mounted Raspberry Pi Zero 2 W with a wide-angle camera that captures a JPEG every 10 seconds. Each frame is sent over Wi-Fi to a central server running YOLOv8 person detection and two MobileNetV2 classifiers that distinguish queued from seated people. Results are stored in Redis (for the current snapshot) and InfluxDB (for time-series history), then served to a Streamlit dashboard.

The practical outcome: students check the dashboard, see the queue is 25 people deep and 80% of seats are taken, and decide to eat 20 minutes later when the crowd has thinned. No app install required, just a browser.

2. Objectives

What it does for the institute

- Gives students real-time visibility into mess congestion before they leave their room.
- Provides mess administration with occupancy data to identify peak hours, plan staffing, and evaluate whether staggered meal timings reduce wait times.
- Generates a historical record of crowd patterns across days and weeks, which currently does not exist.

What we learn from building it

- **End-to-end IoT pipeline construction:** Designing the full path from a physical camera to a live web dashboard, including edge capture, authenticated ingestion, computer vision inference, time-series storage, and a read-only frontend.
- **Computer vision in constrained environments:** Running YOLOv8 detection on a budget single-board computer setup and tuning confidence thresholds against real-world lighting

and angles.

- **Two-stage classification:** Combining spatial zone assignment (is this person physically in the queue area?) with a learned classifier (does this person's posture match someone waiting in line?) rather than relying on either signal alone.
- **Production discipline on a student project:** Authenticated APIs, Pydantic schema validation, Docker Compose for local services, automated tests (63 passing), linting, type checking, and documented integration procedures.

Future possibilities

- **Multi-camera deployment:** The system already supports multiple camera IDs in its configuration. Scaling to mess halls on different floors or separate campus canteens requires only mounting additional Pi units and adding their zone definitions.
- **Push notifications:** Alert students via a Telegram bot or campus app when the queue drops below a threshold.
- **Predictive estimates:** With enough historical data, forecast expected queue length 15-30 minutes ahead based on day-of-week and time-of-day patterns.
- **Menu correlation:** Cross-reference crowd data with the day's menu to quantify which meals drive higher traffic.
- **Waste reduction data:** Combine occupancy trends with food-preparation quantities to help the mess reduce over-preparation on low-attendance days.

3. System Pipeline

The system has three stages: **capture**, **process**, and **display**.

CAPTURE	PROCESS	DISPLAY
+-----+	+-----+	+-----+
Raspberry JPEG POST	Central Server (FastAPI)	GET Streamlit
Pi Zero 2W ----->		<---- Dashboard
+ Camera (auth key)	YOLOv8 person detection	
+-----+		- Headcount
every 10s	v	- Queue len
736x490 px	Zone assignment (polygon)	- Seat occ.
via Wi-Fi		- Crowd lvl
	v	- 60m chart
	MobileNetV2 classifiers	+-----+
	(queue + seated)	
	+---+---+	
	Redis InfluxDB	
	(live) (history)	
	+-----+	

Stage 1 - Capture (Raspberry Pi): A Python script on the Pi calls `libcamera-still` to grab a 736x490 JPEG, pipes it to memory (no SD card writes), and POSTs it to the server with an API key header. This repeats every 10 seconds. The Pi runs headless on Raspberry Pi OS Lite.

Stage 2 - Process (Central Server / FastAPI): The server validates the JPEG, runs YOLOv8n person detection, maps each detected person's foot-point into camera-specific polygon zones (queue, seating, entrance), then runs two MobileNetV2 binary classifiers on person crops to refine the counts. The final metrics (headcount, queue length, seats occupied, occupancy percentage, crowd level) are written to Redis (for instant "current state" reads with a 30-second TTL) and InfluxDB (for persistent history).

Stage 3 - Display (Streamlit Dashboard): A read-only web dashboard polls the API every 8 seconds. It shows live headcount, queue count, seat occupancy, a green/amber/red crowd indicator, and a 60-minute trend chart. It has zero access to database credentials; it reads only from the API's public GET endpoints. An upload mode also lets anyone drop in a photo for offline person-count analysis without needing a live camera.

4. Current Progress

What is built and tested

The complete software pipeline is operational. As of 8 September 2026:

- **63 automated tests pass** covering inference, zone geometry, metrics, API routes, dashboard rendering, classifiers, and the photo-upload workflow.
- **Full live-stack verification** completed: Redis, InfluxDB, FastAPI, YOLOv8, both MobileNetV2 classifiers, and Streamlit dashboard running together, with ingestion producing correct status and history readback.
- The API handled **1,000 concurrent read requests at 700 req/sec** on localhost during load testing.
- Cold photo analysis takes ~3.7 seconds; warm analysis takes ~0.23 seconds (27 people detected in the committed test frame).
- Linting (Ruff), type checking (MyPy across 21 source files), formatting, Docker Compose validation, and shell-syntax checks all pass.

Hardware status

All components have been identified and sourced. Procurement is pending; the Raspberry Pi Zero 2 W and Camera Module 3 Wide will be acquired before final mounting and calibration.

Prototype — Detection in action

The model runs on real mess hall photos and produces bounding boxes colour-coded by zone: red for queue, cyan for seated. Three test images from different mess environments:

Image 1 — Queue detection (12 people detected) A cafeteria queue with 12 persons identified, all classified as queued. Inference time: 2,451 ms. Crowd level: GREEN (0.245).

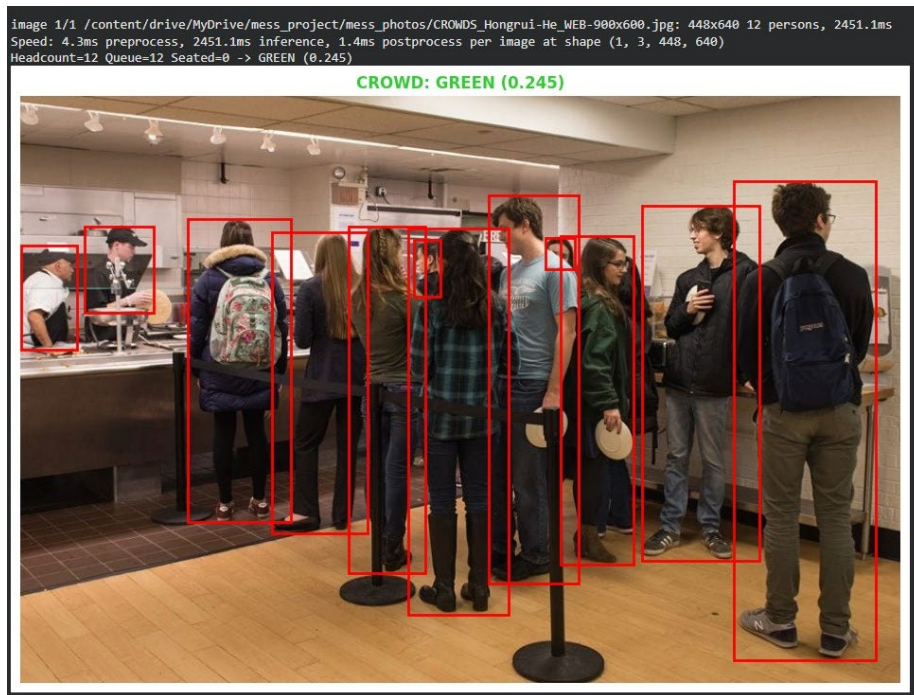


Image 2 — Queue + seating differentiation (17 people detected) An Indian college canteen with both standing and seated students. The model distinguishes 6 people in the queue zone (red) and 12 seated (cyan). Inference time: 4,472 ms. Crowd level: GREEN (0.204).

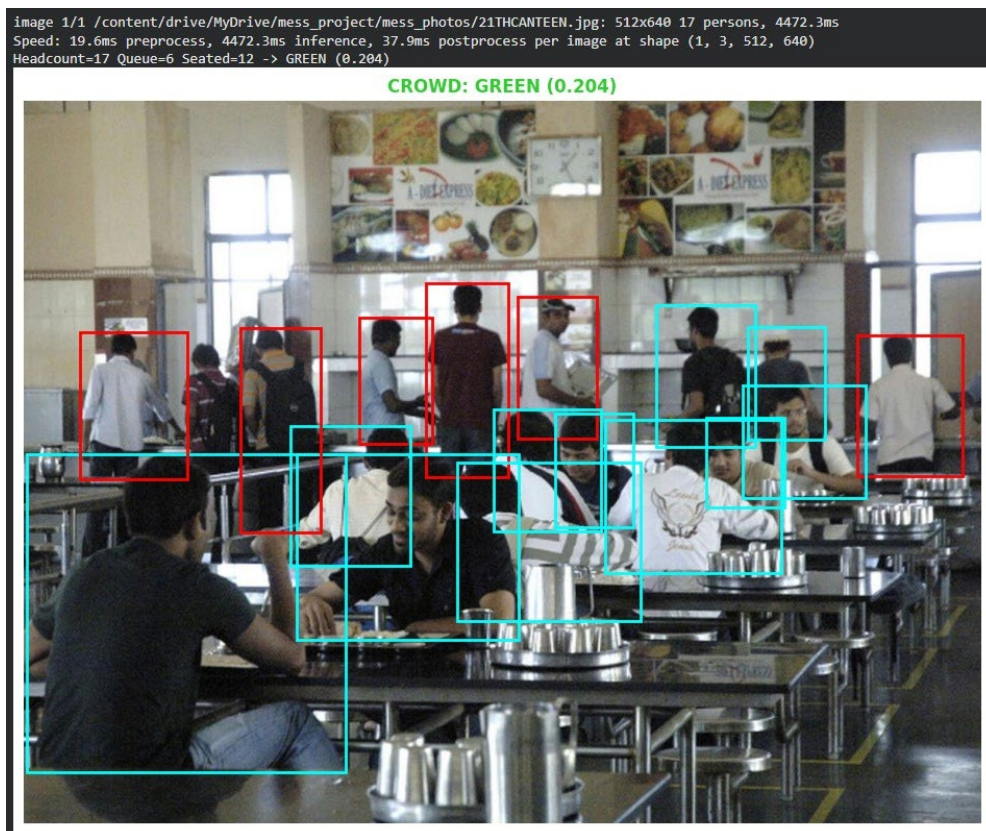
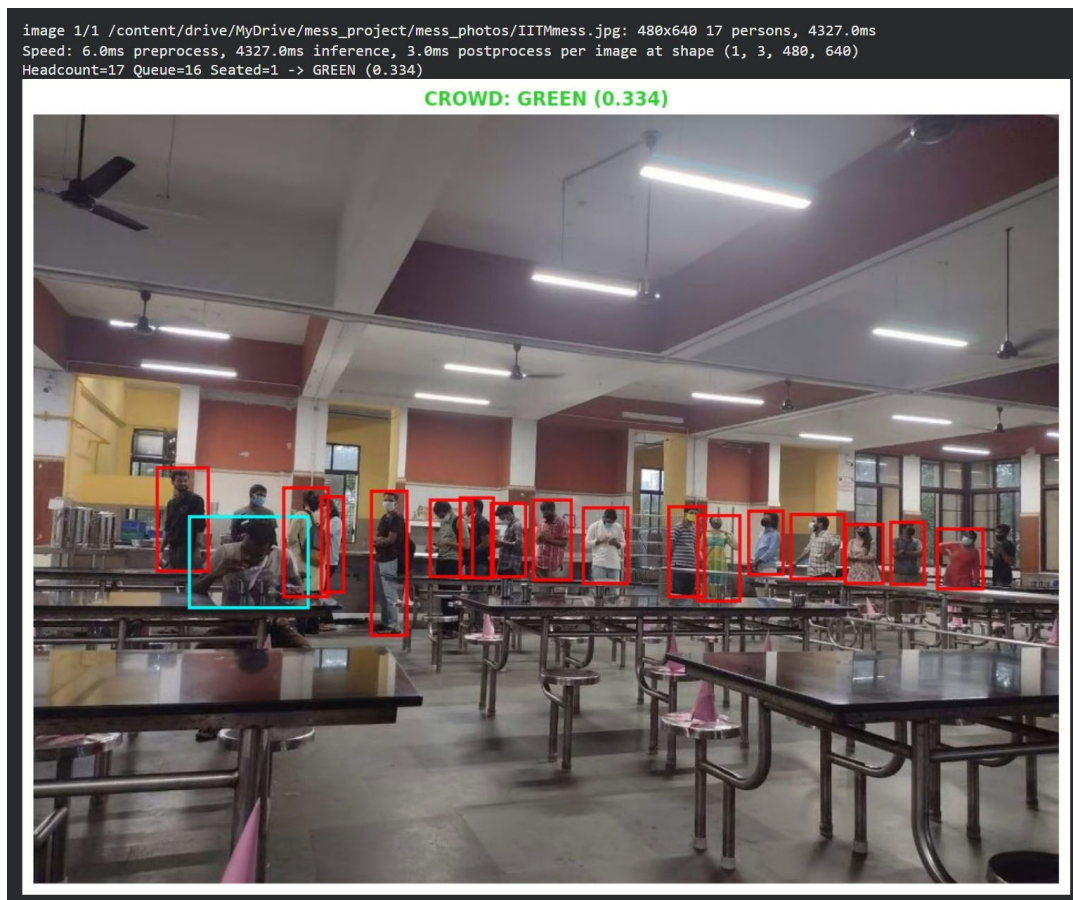
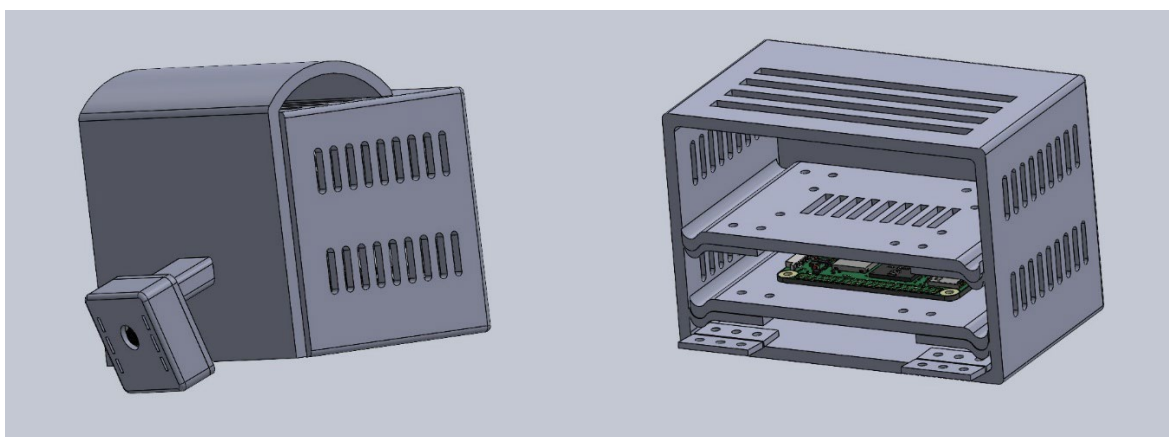


Image 3 — Mess hall queue (17 people detected) A larger IIT mess hall with 17 detections: 16 in the queue zone, 1 seated. Inference time: 4,327 ms. Crowd level: GREEN (0.334).



Enclosure design

A 3D-printed enclosure houses the Pi Zero 2 W, camera module, and battery pack for wall mounting:



Remaining work before deployment

1. Mount the camera at its final position in the mess and retrace the zone polygons against the actual camera viewpoint.
2. Validate classifier accuracy on real footage from the mounted camera (current zones and thresholds are calibrated against a sample image).

3. Provision production Redis and InfluxDB credentials on the hosting server.
4. Configure the Pi's systemd service with the final server URL and API key.
5. Run a 48-hour burn-in to check capture reliability, Wi-Fi reconnection, and battery life under continuous operation.

5. Budget

#	Component	Qty	Price (INR)	Source	Status
1	Raspberry Pi Zero 2 W	1	1,599	Electropi	Pending
2	Raspberry Pi Camera Module 3 Wide	1	3,088	Electropi	Pending
3	DFRobot MP2636 Power Booster and Charger Module	1	862	Robocraze	Pending
4	3000mAh 3.7V LiPo Battery	1	469	Robu	Pending
5	5V 3A Micro-USB Power Supply	1	249	Robu	Pending
	Total		6,267		Pending

All software components (YOLOv8, PyTorch, FastAPI, Streamlit, Redis, InfluxDB) are open-source and free. Cloud hosting costs for production deployment are not included in this table; local development uses Docker Compose with no external charges.

Report prepared September 2026. Source code and full documentation at github.com/the-robotronics-club/MQM.